

[← Go Back](#)

Hardware

Nicla Voice



Tutorials

Door Intruder Detector Using ML with the Nicla Voice

Audio Analysis with Machine Learning and the Nicla Voice

Glass-Breaking Detection Using Edge Impulse & Nicla Voice

Motion Detection with Nicla Voice and Machine Learning Tools

Nicla Voice User Manual

Home / Hardware / Nicla Voice / **Door Intruder Detector Using ML with the Nicla Voice**

Door Intruder Detector Using ML with the Nicla Voice

This application note describes how to identify when intruders are trying to force an entrance door by analyzing surroundings sounds using a Nicla Voice, Machine Learning Tools, and the Arduino Cloud.

Author · Christopher Mendez · Last revision · 07/17/2024

ON THIS PAGE

Introduction

Goals

Hardware and Software Requirements +

The Machine Learning Model

Intruder Detector System Setup

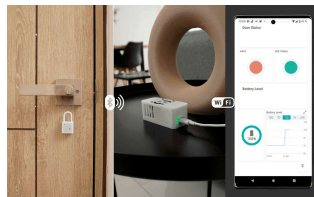
Intruder Detector System Overview +

Full Intruder Detector Example

Conclusion +

Introduction

Security has always been a very important factor for our well-being. From the personal to the material, keeping our assets safe is something we struggle with every day, in our cars, lockers, computers, and our houses, we insure everything is locked up, but there are times when that is not enough.



Cloud-connected Intruder Detector using a Nicla Voice and a Portenta H7

In such a connected world, where we have a lot of information in the palm of our hand, it would also be very useful to have the status of these

[Help](#)

analogous way, such as the front door of our home. Leveraging the integrated microphone of the Nicla Voice provides the possibility to build this kind of protection system.

Goals

The goal of this application note is to showcase an intruder detection and monitoring system for the front door of the house using a combination of Nicla Voice and Portenta H7 as a host, and the Arduino Cloud. The project's objectives are the following:

- Analyze the surrounding sounds with the onboard microphone of the Nicla Voice and run artificial intelligence algorithms at the edge.

- Correctly identify if a door has been opened in an ordinary manner or suffered a forced attempt.

- Report opening and intruder detection events through Bluetooth® Low Energy to a Host.

- The host must forward the data from the Nicla Voice to the Arduino Cloud using an Internet connection (Wi-Fi® in this case).

- Create an Arduino Cloud dashboard that syncs in real time to inform and alert the user.

- Gather real-world data to train a Machine Learning model.

Hardware and Software Requirements



Nicla Voice, Portenta H7,
and 3D enclosures

Hardware Requirements

Nicla Voice

3.7 V LiPo battery

Portenta H7

USB-C® cable

Micro USB cable

5 VDC wall power adapter

DC brushless fan Model:
LD3007MS

[Nicla Voice and Portenta H7](#)

[3D Printed enclosures](#)

Software Requirements

[Arduino IDE 1.8.10+](#), [Arduino IDE 2](#), or [Arduino Cloud Editor](#).

If you are going to use an offline Arduino IDE, you must install the following libraries:

`ArduinoBLE`,

`ArduinoIoTCloud`, and

`Arduino_ConnectionsHandler`.

You can install them using the Arduino IDE Library Manager.

The [intruder detector example code](#).

Arduino Machine Learning Tools

[Edge Impulse® CLI](#), to easily flash any Machine Learning model created with the Arduino Machine Learning Tools, we can install the [Edge Impulse® Cli](#)

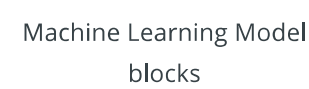
Syntiant® uploader and precompiled ML model.

The Machine Learning is a way of

To be able to identify when the door

First, we flashed the Nicla Voice with

Here is our model design:



As we are using the Nicla Voice, we must follow the model design to be supported by the onboard Syntiant® NDP120 Neural Decision Processor™, taking care of these characteristics.

In the time series data block:

Window size: 968 ms

Window increase: 484 ms

Frequency: 16000 Hz

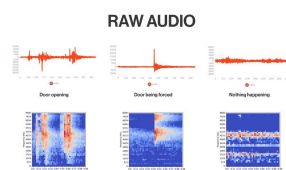
In the processing block:

Audio (Syntiant): This DSP block computes log Mel-filterbank energy features from audio signals.

In the learning block:

Classification: This block includes the Neural Network architecture to learn patterns from audio data.

Here is a graphical representation of how the samples look before and after the digital signal processing:



Raw audio samples and their spectrograms

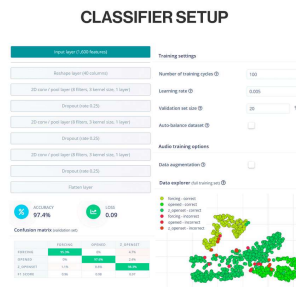
Our three classes are:

Opened: here we record the sound of the door being opened.

Forcing: here we record the sound of the door lock being hit with a metallic tool.

Z_openset: here we record the environment noise when nothing is happening and unknown sounds like music and

Here is the neural network classifier configuration for this application:



Neural Network Configuration

After the model is trained with a lot of samples of a door being opened, being forced and nothing happening (z_openset), we got considerable good results that we can evidentiate on the "Confusion matrix" with a 97.4% of accuracy being achieved with a validation set of new data.



The model performance can be affected if the application is implemented on a very different environment than the one used for training. It's recommended to feed the datasets with new samples and retrain the model for a new and upgraded deployment.

For a new model deployment, use the [Syntiant® uploader](#) and replace the `ei_model.synpkg`, which you can file inside the downloaded folder with yours.

Intruder Detector System Setup

In this application, we don't need any particular wiring diagram other than

The Nicla Voice will be attached to the guarded door and near the lock, the Portenta H7 host will be somewhere with good Wi-Fi® coverage and not so far from the Nicla Voice due to Bluetooth® Low Energy narrow range of a couple of meters.



Nicla Voice and Portenta H7 deployment spots

The Nicla will communicate with the Portenta H7 through Bluetooth® Low Energy advertising any event to the host that will receive and forward the notification to the Arduino Cloud using Wi-Fi®.

The Nicla Voice is powered by a LiPo battery inside its enclosure. The Portenta H7 will be powered by a 5 VDC USB adapter. Although the Portenta H7 can work for long periods without needing external cooling, it is always recommended to keep the devices that are working 24/7 as cool as possible, having the Portenta H7 enclosure a cooling fan connected.

Intruder Detector System Overview

The Nicla Voice attached to the door is running a machine learning model listening to the surrounding noises and searching for known sounds like

example, to simulate a forcing process.

If the sounds are recognized with a certainty higher than 70% the event will be considered as occurred and a Bluetooth® Low Energy notification will be sent to the Portenta H7 host updating the door status. If the sounds are recognized with a certainty higher than 70% the event will be considered as occurred and a Bluetooth® Low Energy notification will be sent to the Portenta H7 host updating the door status.

The Portenta H7 will be in charge of receiving the door event notification and updating the dashboard variables in the IoT Cloud immediately so the user can be aware in real time of any change. In addition to these features, the application dashboard will show a historic record of events, alongside an "Intruder Alert notification", the Bluetooth® Low Energy connection status between both devices and the Nicla Voice battery level so we can know when to recharge it.

Nicla Voice Code

We will go through some important code sections to make this application fully operative. We will begin with the required libraries:

Including `NDP.h` will enable and run the Neural Decision Processor (NDP120), it's included in the BSP of the Nicla Voice.

Including `ArduinoBLE.h` will enable the Bluetooth® Low Energy communication, install it by searching for it on the [Library Manager](#)

The Bluetooth® Low Energy services and characteristics are standardized for the specific use of this application's features. The service is defined as `Immediate alert` which makes it ideal for the use that we will give of notifying on a door opening or forcing event. In addition to this, we defined two characteristics, one for the `Alert Level` that will advertise between both events and a `Battery Level` one to advertise the Nicla Voice battery level to the host. Notice that they have specific and standardized Bluetooth® Low Energy UUIDs.

```

1 #include "NDP.h"
2 #include <ArduinoBLE.h>
3
4 // Alert Service
5 BLEService alertService("
6
7 // BLE Alert Characterist
8 BLEUnsignedCharCharacteri
9
10
11 // Bluetooth® Low Energy
12 BLEUnsignedCharCharacteri
13

```

The Nicla Voice's integrated NDP needs some files stored in the external flash of the board which are:

```

mcu_fw_120_v91.synpkg
dsp_firmware_v91.synpkg
ei_model.synpkg

```



The files must be named exactly as shown above.

[Here](#) you can download these files including the Machine Learning model already trained and the

After downloading the required files, to store them in the Nicla Voice memory you must follow the steps detailed on this [guide section](#).



These files won't be overwritten after any sketch update.

```
1 // Neural Decision Process
2 NDP.begin("mcu_fw_120_v9
3 NDP.load("dsp_firmware_v
4 NDP.load("ei_model.synpk
5 Serial.println("packages
6 NDP.getInfo();
7 Serial.println("Configur
8 NDP.turnOnMicrophone();
```

The main responsibility of the Nicla Voice code is to listen to and identify the trained sounds. The below code section is in charge of comparing the inferred category and taking a certain action between them.

In the case the Nicla detects the door opening, the `label` parameter will turn to `NN0:opened`, this will trigger a Bluetooth® Low Energy alert sending command followed by a listening pause to avoid duplicated alerts and a green LED blinking to visually indicate the event. The same for the `NN0:forcing` label, with the difference that in this case the LED flashes red and the alert message changes.

```

1  /**
2   Inference Interruption
3   it controls the built-
4
5   Possible labels: NN0:(
6
7   Alerts: 1 = mild alert
8
9   @param label The infer
10 */
11 void BLEsend(char* label
12
13 if (strcmp(label, "NN0
14   alertLevel.writeValu
15   NDP.noInterrupts();
16   nicla::leds.begin();
17   nicla::leds.setColor
18   delay(3000);
19   nicla::leds.end();
20   NDP.interrupts();
21 }
22 if (strcmp(label, "NN0
23   alertLevel.writeValu
24   NDP.noInterrupts();
25   nicla::leds.begin();
26   nicla::leds.setColor
27   delay(3000);
28   nicla::leds.end();

```

These methods are used to update the characteristics of this application:

```

1 alertLevel.writeValue(x);
2 batteryLevelChar.writeValu

```

Portenta H7 Code

The Portenta H7 needs the following libraries:

`ArduinoIoTCloud.h` This one handles the Arduino Cloud connection and project variables publishing. It can be installed directly from the Arduino Library Manager.

connection and can be installed directly from the Arduino Library Manager.

`thingProperties.h` This is automatically generated by the Arduino Cloud, however, if you are using an offline IDE verify it's in the same directory as your sketch.

In the global parameters, we have two variables that will store the local status of the alerts and the Nicla Voice battery level:

```
1 #include <ArduinoIoTCloud
2 #include <Arduino_Connect
3 #include "thingProperties
4 #include <ArduinoBLE.h>
5
6 #define DEBUG false
7
8 // Global Parameters
9 byte BatteryValue = 0; /
10 byte AlertValue = 0; /
```

In the `thingProperties.h` file we encounter the definition of several important variables to properly connect our project to the internet and the Arduino Cloud.

We must update the `THING_ID` variable with our Arduino Cloud Thing ID.

For a successful internet connection, we must update the `SSID` and `PASS` variables with our Wi-Fi® credentials.

In the `initProperties()` function we have listed the variables that will be monitored in the IoT Cloud and their update method, in this case, is `ON_CHANGE` which means that the Portenta H7 will only send the value of any variable to the Cloud if it's

```
1  const char THING_ID[] = "  
2  
3  const char SSID[] = "  
4  const char PASS[] = "  
5  
6  String DoorEvent = "Syste  
7  int battery;  
8  bool BLEstatus;  
9  bool alertStatus;  
10  
11 void initProperties(){  
12  
13   ArduinoCloud.setThingId  
14   ArduinoCloud.addPropert  
15   ArduinoCloud.addPropert  
16   ArduinoCloud.addPropert  
17   ArduinoCloud.addPropert  
18  
19 }  
20  
21 WiFiConnectionHandler Ard
```

The main responsibility of the Portenta H7 code is to connect to the Nicla Voice and be aware of new notifications to forward them to the Cloud. To achieve this, the first thing done by the Portenta is to initialize the Bluetooth® Low Energy communication, then initialize the Arduino Cloud service and start scanning for peripherals.

```
1 // Initialize BLE
2 if (!BLE.begin()) {
3     Serial.println("Start
4     while (1) {
5     }
6 }
7
8 // Defined in thingProp
9 initProperties();
10
11 // Connect to Arduino C
12 if (!ArduinoCloud.begin
13     Serial.println("Ardui
14 }
15
16 setDebugMessageLevel(2)
17 ArduinoCloud.printDebug
18
19 // start scanning for
20 BLE.scan();
21 Serial.println("Scannin
```

The scanning process will automatically stop when it finds a "Nicla Lock" called device, then it will search for its services and characteristics with the `NiclaLockHandler()` function.

```

1  void loop() {
2
3      // check if a peripheral
4      BLEDevice peripheral =
5
6      // Turn on the Green LED
7      if (ArduinoCloud.connected) {
8          digitalWrite(LED_GREEN, LOW);
9      } else {
10         digitalWrite(LED_GREEN, HIGH);
11     }
12
13     if (peripheral) {
14         // peripheral discovered
15         Serial.print("Found ");
16         Serial.print(peripheral.name());
17         Serial.print(" ");
18         Serial.print(peripheral.address());
19         Serial.print(" ");
20         Serial.print(peripheral.localName());
21         Serial.println();
22
23         // Check if the peripheral is local
24         if (peripheral.localName() == "ArduinoCloud") {
25             // stop scanning
26             BLE.stopScan();
27
28             // Nicla Voice notification
29             Serial.println("Nicla Voice notification");
30         }
31     }
32 }

```

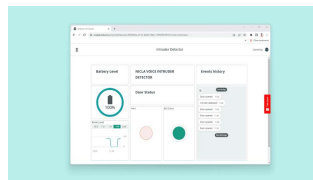
```
1 // discover peripheral a
2 Serial.println("Search
3
4 if (peripheral.discover
5   Serial.println("Serv
6 } else {
7   Serial.println("Attr
8   peripheral.disconnect
9
10  while (1)
11    ;
12    return;
13  }
14
15  // retrieve the simple
16  BLECharacteristic batt
17  BLECharacteristic aler
18
19  // subscribe to the si
20  Serial.println("Subscr
21  if (!batteryLevelChar
22    Serial.println("no s
23    peripheral.disconnect
24    return;
25  } else if (!batteryLev
26    Serial.println("simp
27    peripheral.disconnect
28    return;
```

Finally, the Portenta verifies continuously if a characteristic is updated to upload it to the Cloud:

```
1 // while the peripheral
2 while (peripheral.conr
3
4 // check if the valu
5 if (batteryLevelChar
6
7     batteryLevelChar.r
8
9     Serial.print("Bat
10    Serial.print(Batte
11    Serial.println(" %
12
13    battery = Battery\
14 }
15
16 // check if the valu
17 if (alertLevel.value
18
19     alertLevel.readVal
20
21     Serial.print("Aler
22     Serial.println(Ale
23
24
25     if (AlertValue ==
26         control = !contr
27         if (control) {
28             DoorEvent = "[
```

The Cloud Dashboard

Taking advantage of the Arduino Cloud, we can seamlessly integrate a simple but powerful dashboard to monitor and visualize the status of the system in real-time:

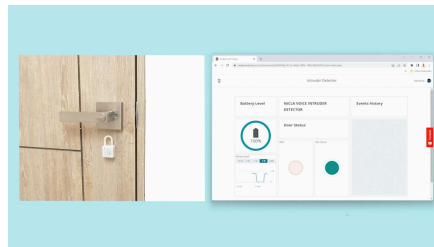


Arduino Cloud project
dashboard

Within the Arduino Cloud's dashboard, the system variables can be monitored. We have a battery

widget to store the events historically, a red LED to shine when an intruder is detected by the system, and a green LED to show the Bluetooth® Low Energy connection status between the Nicla Voice and the Portenta H7 host.

We can easily access this dashboard from a PC, mobile phone or tablet from anywhere, receiving an instantaneous update wherever we are. In addition, we can set different integrations to complement our project, for example, setting up an IFTTT automation to receive an email, a cellphone notification, or even triggering some automation like turning on all the lights at home whenever an alert is fired.



Full Intruder Detector Example

All the necessary files to replicate this application notes can be found below:

The complete code can be downloaded [here](#)

The enclosure 3D printed models can be downloaded [here](#).

The Machine Learning Tools project is public [here](#) so you can clone it and modify it to adapt it to your needs by improving the dataset or model

Conclusion

In this application note, we have learned how to implement a Machine Learning Tools project powered by Edge Impulse and the Arduino Cloud at the edge using a Nicla Voice and a Portenta H7. This application could be a simple demonstration of how Arduino's environment simplifies the workflow for developing smart solutions to solve real-life situations by integrating "complex" and mighty algorithms with just a few easy-to-follow steps. We covered sound recognition, Bluetooth® Low Energy communication, and real-time Cloud monitoring.

Next Steps

As you already know how to develop an Intruder Detector with the Nicla Voice and the Portenta H7, it's time for you to continue exploring all the capabilities of the Arduino Pro environment to integrate it with your professional setup and improve it with powerful solutions.

You can create your application by training your own model with different scenario sounds or using the built-in Nicla Voice's IMU to analyze machine vibrations and making smart your existing manufacturing setup.

Suggest changes	Need support?	License
The content on docs.arduino.cc	Help Center Ask the Arduino Forum	The Arduino documentation is licensed under the

through a
public
[GitHub](#)
repository.
If you see
anything
wrong, you
can edit this
page [here](#).

[Share Alike 4.0](#)
license.

Was this article helpful?

